

fine-tuned model performance, evaluated by both *Eval-Machine* and *Eval-Human*, exhibits a significant accuracy drop at Pass@1. This indicates that the fine-tuned model lacks certainty under greedy decoding, unless one leverages the strength of probabilistic generation. Furthermore, to probe beyond the standard Pass@ k , we introduce a complementary metric $\text{Pass}@k^{(m)}$, which measures the probability that *at least* m of the top- k generated outputs are correct. Note that $\text{Pass}@k^{(1)}$ reduces to the classic Pass@ k evaluation. As shown in Fig. 2(b), the accuracy under $\text{Pass}@k^{(m)}$ decreases sharply as m increases, revealing that even when allowing multiple generations (e.g., $k = 10$), most outputs are still incorrect; for instance, $m = 5$ requires half of the generations to succeed, yet performance drops drastically in this setting.

Problem statement. As shown in Fig. 2, achieving effective code fine-tuning for LLMs is far from trivial, with root causes largely overlooked in prior work. To address this gap, we focus on two key questions in this study: **(Q1)** *What causes the difficulty of effective code fine-tuning for LLMs?* **(Q2)** *How can the existing fine-tuning protocol in (1) be advanced to achieve improved effectiveness?*

In the rest of the paper, we address (Q1) by identifying the “memorization barrier” in LLM code fine-tuning, where excessive memorization of fine-tuning data by the pretrained LLM can *hinder* effective adaptation (see Sec. 4). We then address (Q2) by tackling this barrier through an information bottleneck perspective, enforcing *equality* across data with different memorization levels so that the fine-tuner learns from the most informative aspects of the data (see Sec. 5).

4 The “Memorization Barrier” in LLM Code Fine-tuning

In this section, we identify and formalize the “*memorization barrier*”, introduced here for the first time, as a potential root cause of the challenges in code fine-tuning.

The memorization lens on (in)effective code fine-tuning. As illustrated in Fig. 2, the code fine-tuned LLM does not appear to gain much from the code dataset, as evidenced by its sharp accuracy drop in Pass@1 and in $\text{Pass}@k^{(m)}$ for large m . This raises the question of whether the limitation stems from the quality of the data themselves or from the fine-tuning process failing to fully exploit the code data. In this work, we assume no control over data curation and instead focus on improving the fine-tuning process.

Our hypothesis is that effective code fine-tuning should enable the model to acquire new domain knowledge beyond pretraining, which motivates a deeper examination of the relationship between fine-tuning data and the pretrained LLM. We approach this from the perspective of *memorization* (Shi et al., 2023; Biderman et al., 2023; Leybzson & Kervadec, 2024; Kiyomaru et al., 2024): Given that the pretraining corpus is massive and may partially overlap with the target domain, we examine whether a non-trivial portion of the fine-tuning data has already been memorized, and how this entanglement, ignored in the conventional fine-tuning protocol 1, underlie the observed limitations.

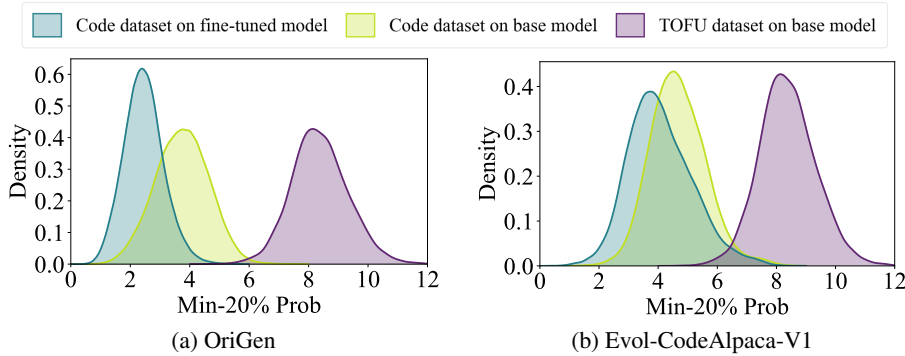


Figure 3: Downstream data already memorized during pretraining. Memorization patterns measured by Min-K% Prob ($K = 20$) using DeepSeek-Coder-7b-Instruct-v1.5 and Llama-3 8B as base models on two code datasets, OriGen and Evol-CodeAlpaca-V1. The x-axis shows Min-20% Prob scores and the y-axis their density. For each dataset, three distributions are reported: $p_0(\mathcal{D}_{\text{code}}; \theta_0)$ (code dataset on the base model), $p_1(\mathcal{D}_{\text{code}}; \theta_{\text{code}})$ (code dataset on the fine-tuned model), and $p_2(\mathcal{D}_{\text{TOFU}}; \theta_0)$ (TOFU dataset on the base model), where p_2 provides the weakest-memorization reference.

Through the lens of memorization, Fig. 3 shows the memorization scores of the fine-tuning code datasets OriGen (Cui et al., 2024) and Evol-CodeAlpaca-V1 (Luo et al., 2023) with respect to the base model (θ_0). Here, memorization is assessed using the Min-K% Prob method (Shi et al., 2023), which measures how likely a sequence has been memorized by a model. It selects the $K\%$ tokens with the lowest predicted probabilities and computes their average negative log-likelihood (i.e., the prediction loss over the least likely tokens). A lower Min-K% Prob score indicates

stronger memorization, as even the least likely tokens are predicted confidently, whereas a higher score suggests weaker memorization.

In Fig. 3, we present the distribution of $\text{Min-K\% Prob}$ (with $K = 20$) scores for the fine-tuning code dataset (OriGen or Evol-CodeAlpaca-V1, denoted $\mathcal{D}_{\text{code}}$) on the pretrained model (θ_0) and its fine-tuned counterpart (θ_{code}), denoted as $p_0(\mathcal{D}_{\text{code}}; \theta_0)$ and $p_1(\mathcal{D}_{\text{code}}; \theta_{\text{code}})$, respectively. For comparison, we also include $p_2(\mathcal{D}_{\text{TOFU}}; \theta_0)$, the distribution obtained from the base model evaluated on the fictitious TOFU dataset Maini et al. (2024), which is excluded from pretraining as it contains synthetic author profiles. The rationale is that $p_2(\mathcal{D}_{\text{TOFU}}; \theta_0)$ serves as a *reference distribution for the weakest memorization* of the base model, since $\mathcal{D}_{\text{TOFU}}$ contains entirely fictitious information. As shown in Fig. 3, p_0 lies to the right of p_1 , reflecting the stronger memorization of $\mathcal{D}_{\text{code}}$ by the fine-tuned model θ_{code} . This is expected, since θ_{code} is directly trained on $\mathcal{D}_{\text{code}}$. More interestingly, *however*, p_0 has shown substantial overlap with p_1 , suggesting that *the base model has already memorized much of the dataset prior to fine-tuning*. This strong pre-existing memorization becomes especially evident when contrasted with the weak memorization reference p_2 .

The above reveals a *new memorization phenomenon* identified from LLM code fine-tuning: fine-tuning data points may already be strongly memorized by the pretrained base model even before fine-tuning. We remark that this finding differs from prior literature, which has typically examined memorization from a privacy perspective (to detect leakage), either by evaluating an LLM against its pretraining data (Carlini et al., 2022) or by assessing a fine-tuned LLM against its fine-tuning data (Zeng et al., 2023). In contrast, ours introduces a *cross-setting* perspective: evaluating the memorization of *fine-tuning datasets* on the *base model*.

“Memorization barrier” in code fine-tuning. Given the strong memorization of code fine-tuning data by the pretrained base model, we posit that this memorization underlies the limited effectiveness of LLM code fine-tuning. This challenge is analogous to a known challenge in nonconvex optimization, “*escaping bad local optima*” (Ge et al., 2015; Criscitiello & Boumal, 2019). Our rationale is that, due to the memorization, the base model becomes trapped in a bad “local optimum” characterized by high memorization. Consequently, the conventional fine-tuning objective (1) struggles to escape this state and converge to a better solution, *i.e.*, a code fine-tuned model with improved generalization. We refer to this issue as **“memorization barrier”** and state it below.

Memorization barrier: The phenomenon where LLM code fine-tuning starts from a base model θ_0 that already strongly memorizes the fine-tuning set $\mathcal{D}_{\text{code}}$, placing optimization in a state the conventional fine-tuning objective struggles to escape, thereby leading to poor generalization on downstream code tasks.

Fig. 4 validates the memorization barrier by comparing $\text{Pass}@k$ performance of the fine-tuned model using the conventional approach (1) with different data pruning ratios, where the most memorized data points (identified in Fig. 3) are removed from the fine-tuning set. We examine whether excluding highly memorized fine-tuning data, *i.e.*, alleviating the memorization barrier at the data source, enables the conventional fine-tuning approach to yield a better code model. The results reveal a striking pattern: removing as little as the most memorized 10% of data yields a substantial improvement in $\text{Pass}@1$ on Eval-Human and Eval-Machine. Further, using fewer fine-tuning data points does not degrade generalization under $\text{Pass}@k$ for large k . These suggest that memorized data act as a *memorization barrier*, capping the effectiveness of fine-tuning from the base model.

The memorization barrier reveals a new effect of memorization on the degraded generalization of LLM fine-tuning. This is different from prior studies, which have primarily examined its negative impact in the context of data contamination (Deng et al., 2024; Dong et al., 2024; Golchin & Surdeanu, 2025; Wang et al., 2025; Riddell et al., 2024), where test samples appear in the pretraining corpus, creating a false impression of “superior” generalization. To tackle this memorization barrier problem, dataset pruning based on memorization appears to be a straightforward way to improve code fine-tuning as shown in Fig. 4. However, this approach is *inefficient*: (1) it requires data-wise attribution to estimate memorization, which is difficult to scale since attribution itself is computationally intensive; and (2) determining the optimal pruning ratio for improved generalization is nontrivial, as exploring all possible choices requires multiple fine-tuning runs, making the process impractical. In the next section, we address the memorization barrier using the *information bottleneck* principle, which eliminates the need for additional data attribution and pruning.

5 Information Bottleneck–Guided LLM Code Fine-tuning

In this section, we address the memorization barrier in LLM code fine-tuning by leveraging the information bottleneck (IB) as a principled form of regularization. The IB framework reduces redundant memorization, preserves task-relevant information, and reshapes representations to foster improved generalization.

IB regularization to overcome the memorization barrier. To address the memorization barrier identified in Sec. 4, we adopt the IB principle (Tishby et al., 2000; Lei et al., 2025; Liu et al., 2024b; Chen et al., 2024) as a remedy that

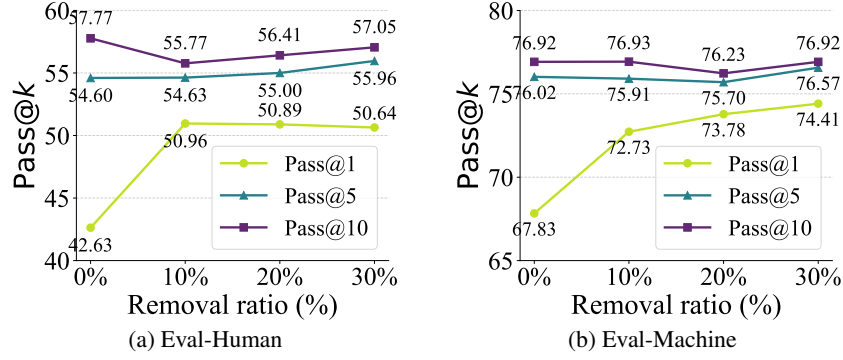


Figure 4: Pass@ k performance of the fine-tuned model on the OriGen dataset using DeepSeek-Coder-7b-Instruct-v1.5, evaluated by (a) *Eval-Human* and (b) *Eval-Machine* (Liu et al., 2023), with different pruning ratios of highly memorized fine-tuning data identified in Fig. 3. The x-axis shows the removal ratio of the most memorized examples (0% corresponds to full dataset), and the y-axis reports Pass@ k for $k \in \{1, 5, 10\}$.

requires no additional data attribution or pruning, enabling an automatic fine-tuning protocol without assuming prior knowledge of the barrier. Motivated by the analogy between the memorization barrier and the problem of *escaping bad local optima* (Ge et al., 2015; Criscitiello & Boumal, 2019) discussed in Sec. 4, we note that the latter can often be mitigated by *random re-initialization* in nonconvex optimization (Lee et al., 2016). Random initialization can be understood as restarting the optimization process, providing an equal opportunity to create a new optimization path and escape bad local optima.

Therefore, similar to random re-initialization, the rationale behind using IB is to transform differently memorized data into equally treated ones by compressing representations to discard spurious information while preserving task-relevant signals for prediction (Tishby et al., 2000). More concretely, the IB principle provides a framework for learning representations that are both predictive and compressed. Given input X , target Y , and representation Z , IB seeks Z that preserves task-relevant information about Y while discarding redundant details from X . This is formulated as

$$\underset{p(z|x)}{\text{minimize}} \quad I(X; Z) - \beta I(Z; Y), \quad (2)$$

where $p(z|x)$ gives the conditional distribution of the latent representation Z given the input X , and $I(\cdot; \cdot)$ denotes mutual information and $\beta > 0$ balances prediction against compression.

The compression term $I(X; Z)$ penalizes how much bottleneck representation Z memorizes input-specific details. In LLMs, direct computation is intractable, so we follow the variational IB framework (Alemi et al., 2017):

$$I(X; Z) \leq \mathbb{E}_{x \sim \mathcal{D}_{\text{code}}} [D_{\text{KL}}(q_{\phi}(z|h_{\theta}(x)) \| p(z))], \quad (3)$$

where $\mathcal{D}_{\text{code}}$ denotes the dataset, D_{KL} is the Kullback–Leibler divergence, $p(z)$ is a simple prior (e.g., $\mathcal{N}(0, I)$), $h_{\theta}(x)$ denotes the hidden representation of input x extracted from a designated intermediate LLM layer (e.g., layer 20), and q_{ϕ} is the variational encoder (with the learnable parameters ϕ) that produces the representation Z given $h_{\theta}(x)$. The design of q_{ϕ} follows the standard variational IB setting (Alemi et al., 2017).

Minimizing this divergence discourages Z from encoding exact input patterns, thereby suppressing memorization of rare or spurious sequences. Hence, from (3), the compression loss is given by

$$\ell_{\text{IB}}^{\text{compress}}(\theta, \phi) = \mathbb{E}_{x \sim \mathcal{D}_{\text{code}}} [D_{\text{KL}}(q_{\phi}(z|h_{\theta}(x)) \| p(z))], \quad (4)$$

In addition, the prediction term $I(Z; Y)$ in (2) prevents over-compression by ensuring that the bottleneck preserves information relevant for predicting Y . This term can be expressed by the log-likelihood of predicting Y from Z ,

$$\ell_{\text{IB}}^{\text{predict}}(\theta, \phi) = \mathbb{E}_{(x,y) \sim \mathcal{D}_{\text{code}}} [\log p_{\theta}(y | z)], \quad (5)$$

where z is sampled from $q_{\phi}(z|h_{\theta}(x))$.

Integrating (4) with (5) based on (2), the proposed IB regularization loss is defined as

$$\ell_{\text{IB}}(\theta, \phi) = \ell_{\text{IB}}^{\text{compress}}(\theta, \phi) - \beta \ell_{\text{IB}}^{\text{predict}}(\theta, \phi), \quad (6)$$